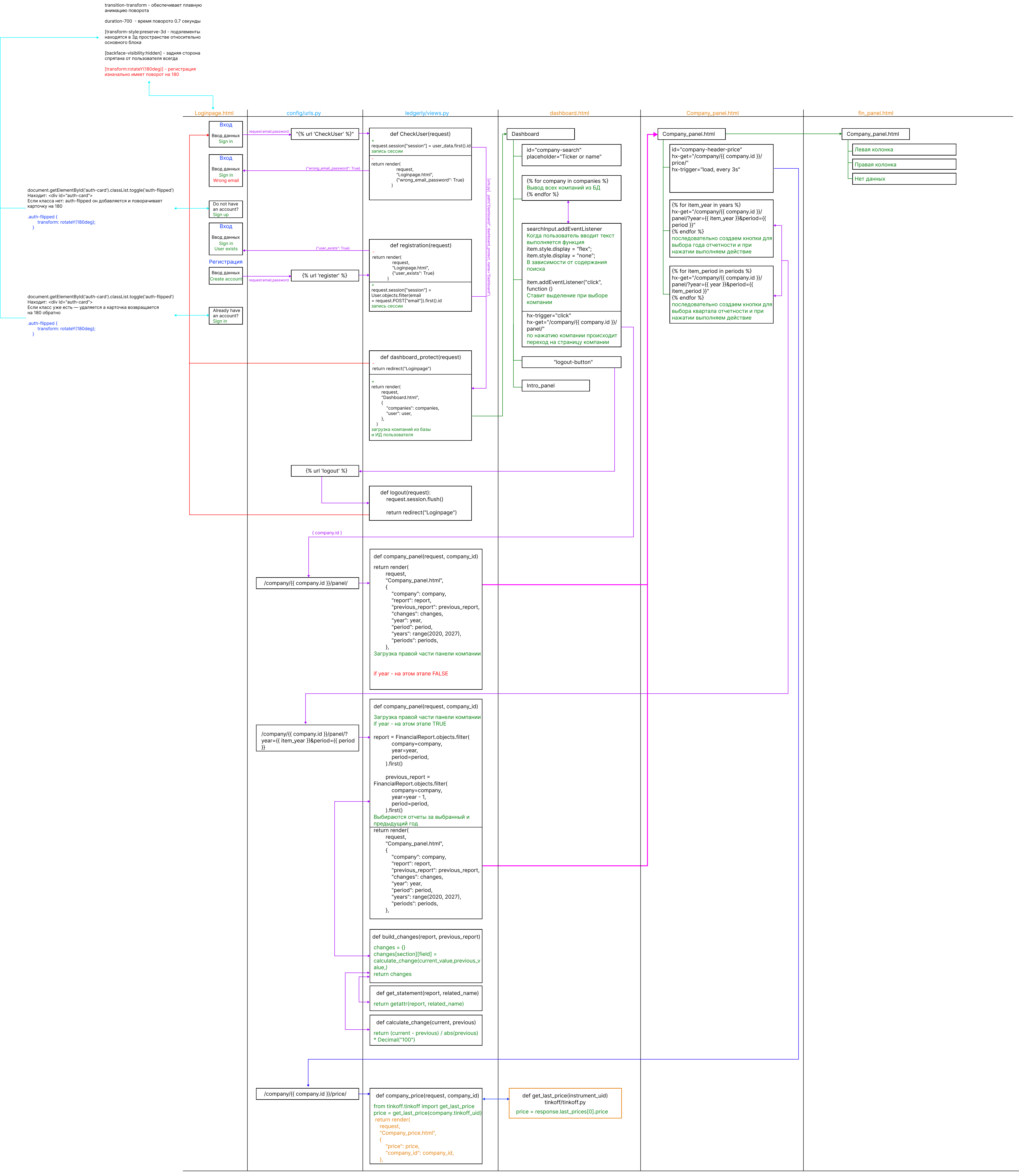


transition-transform - обеспечивает плавную анимацию поворота

duration-700 - время поворото 0.7 секунды

[transform-style:preserve-3d - подэлементы находятся в 3д пространстве относительно основного блока

[backface-visibility:hidden] - задняя сторона спрятана от пользователя всегда

[transform:rotateY(180deg)] - регистрация изначально имеет поворот на 180

document.getElementById('auth-card').classList.toggle('auth-flipped')
Находит: <div id="auth-card">
Если класса нет: auth-flipped он добавляется и поворачивает карточку на 180

```
.auth-flipped {
      transform: rotateY(180deg);
   }
```

document.getElementById('auth-card').classList.toggle('auth-flipped')
Находит: <div id="auth-card">
Если класс уже есть — удаляется и карточка возвращается на 180 обратно

```
.auth-flipped {
      transform: rotateY(180deg);
   }
```

**Loginpage.html**

Вход
Ввод данных
Sign in

Вход
Ввод данных
Sign in
Wrong email

Do not have an account?
Sign up

Вход
Ввод данных
Sign in
User exists

Регистрация
Ввод данных
Create account

Already have an account?
Sign in

**config/urls.py**

request:email,password

"{% url 'CheckUser' %}"

{"wrong_email_password": True}

{"user_exists": True}

request:email,password

{% url 'register' %}

{% url 'logout' %}

{ company.id }

/company/{{ company.id }}/panel/

/company/{{ company.id }}/panel/?year={{ item_year }}&period={{ period }}

/company/{{ company.id }}/price/

urls.py | path("Dashboard/", dashboard_protect, name="Dashboard"),

**ledgerly/views.py**

```
def CheckUser(request)
+
request.session["session"] = user_data.first().id
```
запись сессии

```
-
return render(
        request,
        "Loginpage.html",
        {"wrong_email_password": True}
    )
```

```
def registration(request)
-
return render(
    request,
    "Loginpage.html",
    {"user_exists": True}
  )
+
request.session["session"] =
User.objects.filter(email
= request.POST["email"]).first().id
```
запись сессии

```
def dashboard_protect(request)
-
return redirect("Loginpage")
+
return render(
    request,
    "Dashboard.html",
    {
      "companies": companies,
      "user": user,
    },
  )
```
загрузка компаний из базы
и ИД пользователя

```
def logout(request):
    request.session.flush()

    return redirect("Loginpage")
```

```
def company_panel(request, company_id)

return render(
    request,
    "Company_panel.html",
    {
      "company": company,
      "report": report,
      "previous_report": previous_report,
      "changes": changes,
      "year": year,
      "period": period,
      "years": range(2020, 2027),
      "periods": periods,
    },
```
Загрузка правой части панели компании

if year - на этом этапе FALSE

```
def company_panel(request, company_id)
```
Загрузка правой части панели компании
if year - на этом этапе TRUE

```
report = FinancialReport.objects.filter(
      company=company,
      year=year,
      period=period,
    ).first()

    previous_report =
FinancialReport.objects.filter(
      company=company,
      year=year - 1,
      period=period,
    ).first()
```
Выбираются отчеты за выбранный и предыдущий год

```
return render(
    request,
    "Company_panel.html",
    {
      "company": company,
      "report": report,
      "previous_report": previous_report,
      "changes": changes,
      "year": year,
      "period": period,
      "years": range(2020, 2027),
      "periods": periods,
    },
```

```
def build_changes(report, previous_report)

changes = {}
changes[section][field] =
calculate_change(current_value,previous_value,)
return changes
```

```
def get_statement(report, related_name)

return getattr(report, related_name)
```

```
def calculate_change(current, previous)

return (current - previous) / abs(previous)
* Decimal("100")
```

```
def company_price(request, company_id)

from tinkoff.tinkoff import get_last_price
price = get_last_price(company.tinkoff_uid)
return render(
   request,
   "Company_price.html",
   {
     "price": price,
     "company_id": company_id,
   },
```

```
def get_last_price(instrument_uid)
tinkoff/tinkoff.py

price = response.last_prices[0].price
```

**dashboard.html**

Dashboard

id="company-search"
placeholder="Ticker or name"

{% for company in companies %}
Вывод всех компаний из БД
{% endfor %}

searchInput.addEventListener
Когда пользователь вводит текст выполняется функция
item.style.display = "flex";
item.style.display = "none";
В зависимости от содержания поиска

item.addEventListener("click", function ()
Ставит выделение при выборе компании

hx-trigger="click"
hx-get="/company/{{ company.id }}/panel/"
по нажатию компании происходит переход на страницу компании

"logout-button"

Intro_panel

**Company_panel.html**

Company_panel.html

id="company-header-price"
hx-get="/company/{{ company.id }}/price/"
hx-trigger="load, every 3s"

{% for item_year in years %}
hx-get="/company/{{ company.id }}/panel/?year={{ item_year }}&period={{ period }}"
{% endfor %}
последовательно создаем кнопки для выбора года отчетности и при нажатии выполняем действие

{% for item_period in periods %}
hx-get="/company/{{ company.id }}/panel/?year={{ year }}&period={{ item_period }}"
{% endfor %}
последовательно создаем кнопки для выбора квартала отчетности и при нажатии выполняем действие

**fin_panel.html**

Company_panel.html

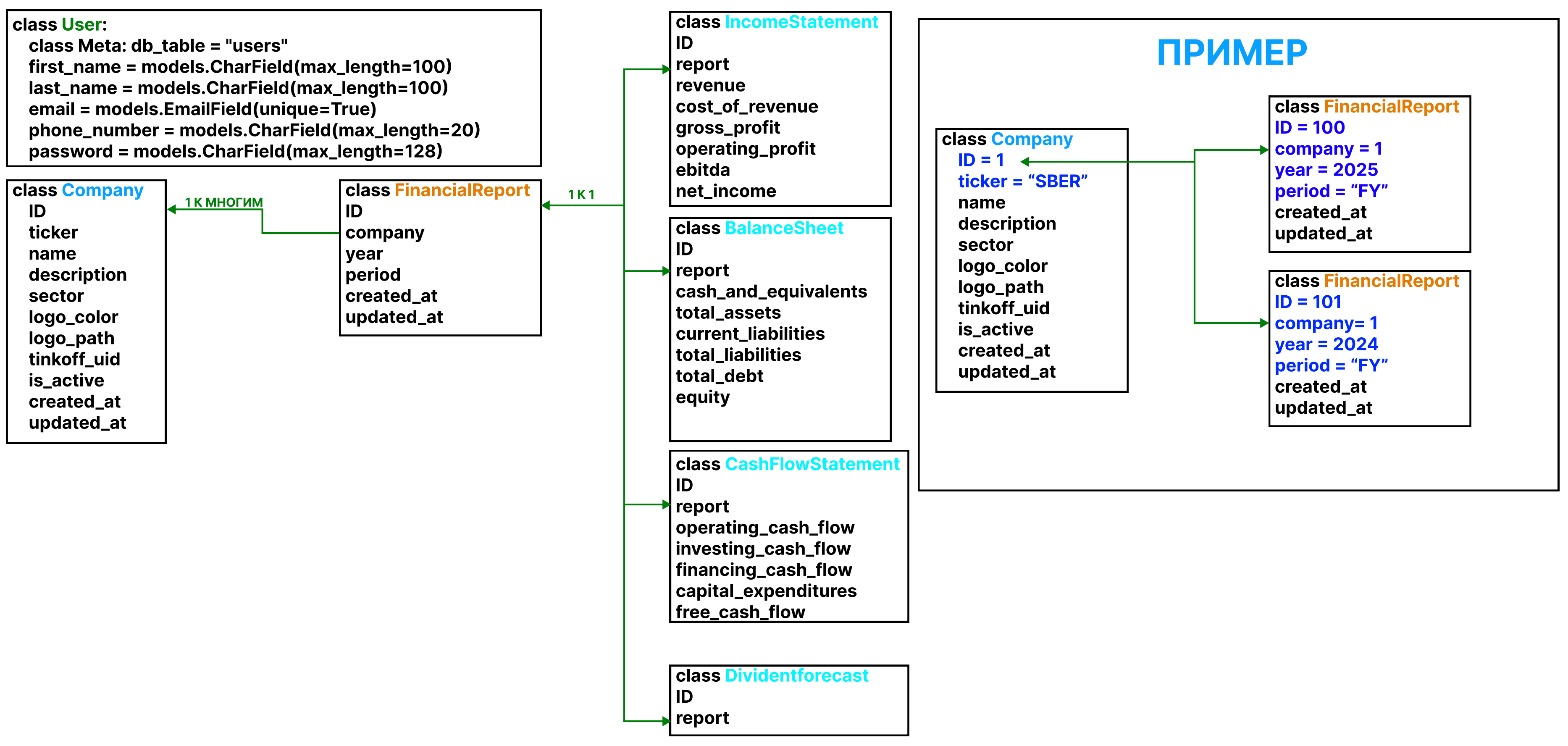
Левая колонка

Правая колонка

Нет данных

# ADMIN

# DB



**class User:**
```
  class Meta: db_table = "users"
  first_name = models.CharField(max_length=100)
  last_name = models.CharField(max_length=100)
  email = models.EmailField(unique=True)
  phone_number = models.CharField(max_length=20)
  password = models.CharField(max_length=128)
```

**class Company**
- ID
- ticker
- name
- description
- sector
- logo_color
- logo_path
- tinkoff_uid
- is_active
- created_at
- updated_at

1 К МНОГИМ

**class FinancialReport**
- ID
- company
- year
- period
- created_at
- updated_at

1 К 1

**class IncomeStatement**
- ID
- report
- revenue
- cost_of_revenue
- gross_profit
- operating_profit
- ebitda
- net_income

**class BalanceSheet**
- ID
- report
- cash_and_equivalents
- total_assets
- current_liabilities
- total_liabilities
- total_debt
- equity

**class CashFlowStatement**
- ID
- report
- operating_cash_flow
- investing_cash_flow
- financing_cash_flow
- capital_expenditures
- free_cash_flow

**class Dividentforecast**
- ID
- report

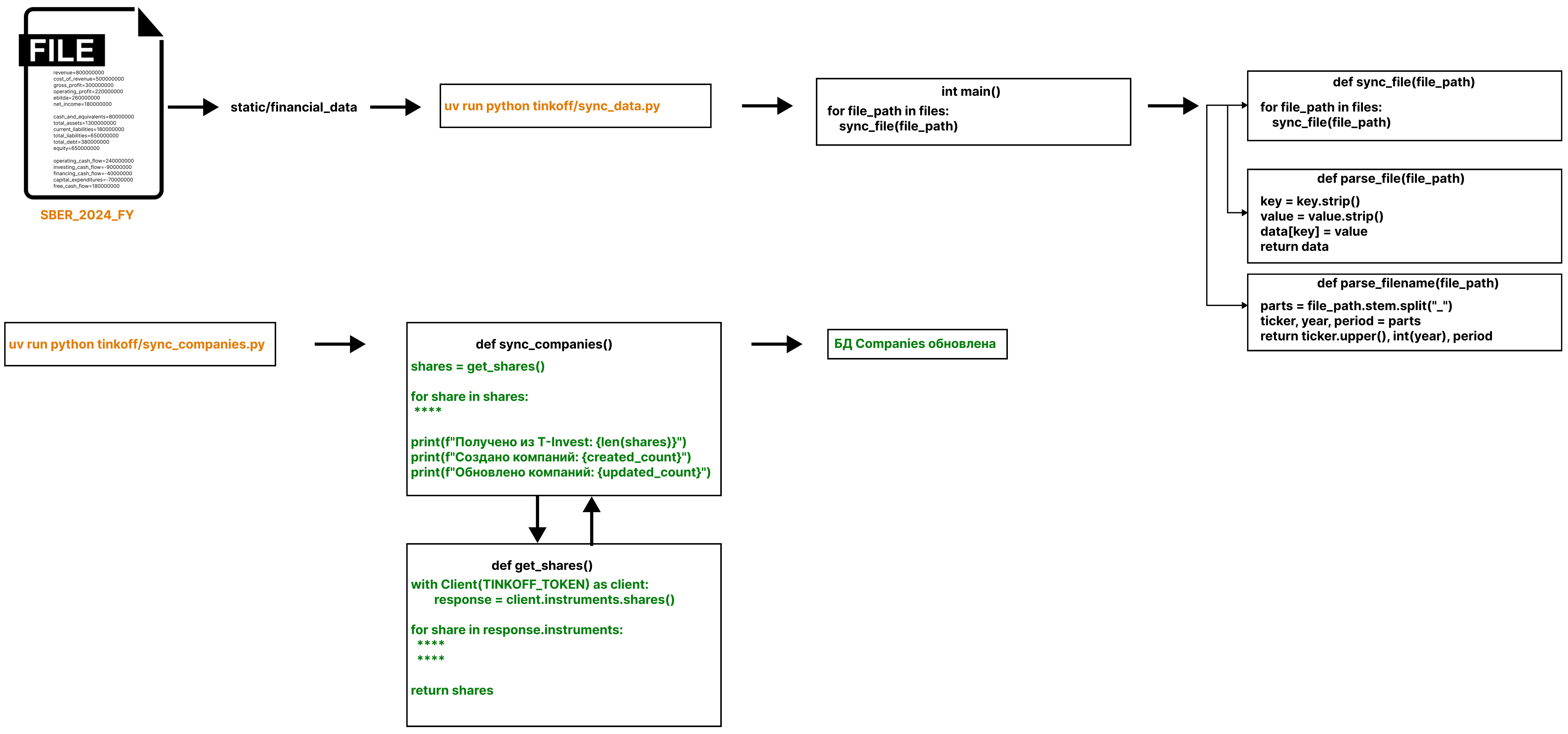
**ПРИМЕР**

**class Company**
- ID = 1
- ticker = “SBER”
- name
- description
- sector
- logo_color
- logo_path
- tinkoff_uid
- is_active
- created_at
- updated_at

**class FinancialReport**
- ID = 100
- company = 1
- year = 2025
- period = “FY”
- created_at
- updated_at

**class FinancialReport**
- ID = 101
- company= 1
- year = 2024
- period = “FY”
- created_at
- updated_at

# Синхронизация базы данных



FILE

```
revenue=800000000
cost_of_revenue=500000000
gross_profit=300000000
operating_profit=220000000
ebitda=260000000
net_income=180000000

cash_and_equivalents=80000000
total_assets=1300000000
current_liabilities=180000000
total_liabilities=650000000
total_debt=380000000
equity=650000000

operating_cash_flow=240000000
investing_cash_flow=-90000000
financing_cash_flow=-40000000
capital_expenditures=-70000000
free_cash_flow=180000000
```

SBER_2024_FY

static/financial_data

`uv run python tinkoff/sync_data.py`

```
int main()
for file_path in files:
   sync_file(file_path)
```

```
def sync_file(file_path)

for file_path in files:
   sync_file(file_path)
```

```
def parse_file(file_path)

key = key.strip()
value = value.strip()
data[key] = value
return data
```

```
def parse_filename(file_path)

parts = file_path.stem.split("_")
ticker, year, period = parts
return ticker.upper(), int(year), period
```

`uv run python tinkoff/sync_companies.py`

```
def sync_companies()
shares = get_shares()

for share in shares:
 ****

print(f"Получено из T-Invest: {len(shares)}")
print(f"Создано компаний: {created_count}")
print(f"Обновлено компаний: {updated_count}")
```

```
def get_shares()
with Client(TINKOFF_TOKEN) as client:
     response = client.instruments.shares()

for share in response.instruments:
  ****
  ****

return shares
```

БД Companies обновлена